tRPC-Agent-Go：构建智能AI应用的Go语言Agent框架

腾讯技术工程 2025年10月13日 02:36 广东 16人

导语

tRPC是腾讯内部覆盖最广的RPC开发框架，覆盖了腾讯大多数微服务，服务了如QQ，腾讯视频，腾讯音乐，QQ浏览器，腾讯新闻，元宝，电脑、手机管家，腾讯云等业务。其中tRPC-Go是使用最广的，累计覆盖将近200w+节点，5w+服务。

2023年起，LLM发展迅速，而Agent开发框架成为连接AI能力与业务应用的重要基础设施。为了便于用户更好的开发Agent应用，并方便和tRPC微服务结合提供高性能，高可用的服务，tRPC-Go在25年持续深耕Agent能力建设，本文将为你全面解读trpc-agent-go(https://github.com/trpc-group/trpc-agent-go)如何构建智能AI应用。

一、技术选型和定位

1.业界框架技术路线分析

目前AI Agent 应用开发框架主要分为两大技术路线：自主多Agent框架和编排式框架。

1.1 自主多Agent框架

自主多Agent框架体现了真正的Agent（Autonomous Agent）理念，每个Agent都具备环境感知、自主决策和动作执行能力。多个Agent通过消息传递和协商机制实现分布式协作，能够根据环境变化动态调整策略，展现出智能涌现特性。

- **AutoGen（Microsoft）**：多Agent协作系统，支持Agent角色专业化和动态协商
- **ADK（Google Agent Development Kit）**：提供完整的Agent生命周期管理和多Agent编排能力
- **CrewAI**：面向任务的多Agent协作平台，强调角色定义和责任链模式
- **Agno**：轻量级高性能Agent框架，专注于多模态能力和团队协作

1.2 编排式框架

编排式框架采用工作流思维，通过预定义的流程图或状态机来组织LLM调用和组件交互。虽然整个系统表现出"智能"特征，但其执行路径是确定性的，更像是"智能化的工作流"，而非真正的自主Agent。

- **LangChain**：基于Chain抽象的组件编排框架，通过预定义执行链路构建LLM应用
- **LangGraph**：有向无环图（DAG）状态机框架，提供确定性状态转换和条件分支

2.两种框架类型技术对比

对比维度	自主多Agent框架	编排式框架
控制模式	分布式自治决策，Agent间协商	集中式流程编排，确定性执行
适用场景	开放域问题求解、创造性任务、多专业协作	结构化业务流程、数据处理管道、标准化作业
扩展方式	水平扩展Agent角色，垂直增强Agent能力	节点扩展和流程图复杂化
执行可预测性	涌现行为，结果多样性高	确定性执行，结果可复现
系统复杂度	Agent交互复杂，调试困难	流程清晰，易于调试和监控
技术实现	基于消息传递和对话协议	基于状态机和有向图执行

3.tRPC-Agent-Go技术定位

行业与生态现状：随着LLM能力的持续突破，Agent开发框架正成为AI应用开发的重要趋势。当前主流的自主多Agent框架（如AutoGen、CrewAI、ADK、Agno等）主要基于Python生态构建，为Python开发者提供了丰富的选择。然而，Go语言凭借其卓越的并发性能、内存安全和部署便利性，在微服务架构中占据重要地位。目前较为成熟的Go语言AI开发框架大多数专注于编排式架构，主要适用于结构化业务流程，现代LLM在复杂推理、动态决策方面能力显著提升，自主多Agent框架相比编排式框架具有以下特征：

- **自适应性**：Agent基于上下文动态调整决策策略和执行路径
- **协作涌现**：多Agent通过消息传递实现去中心化协商和任务分解
- **认知集成**：深度整合LLM的推理、规划、反思能力形成智能决策链路

在我们的业务调研中，腾讯的业务对上面的特性有较强的需求。

因此tRPC-Agent-Go框架选型偏向于自主多Agent协作的模式和架构。但又存在如下问题

1，对于明确的场景和流程，AI工作流更合适，能够给出更稳定的输出

2，腾讯内部已有大量的AI工作流，基于tRPC的开发经验，需要对存量进行兼容

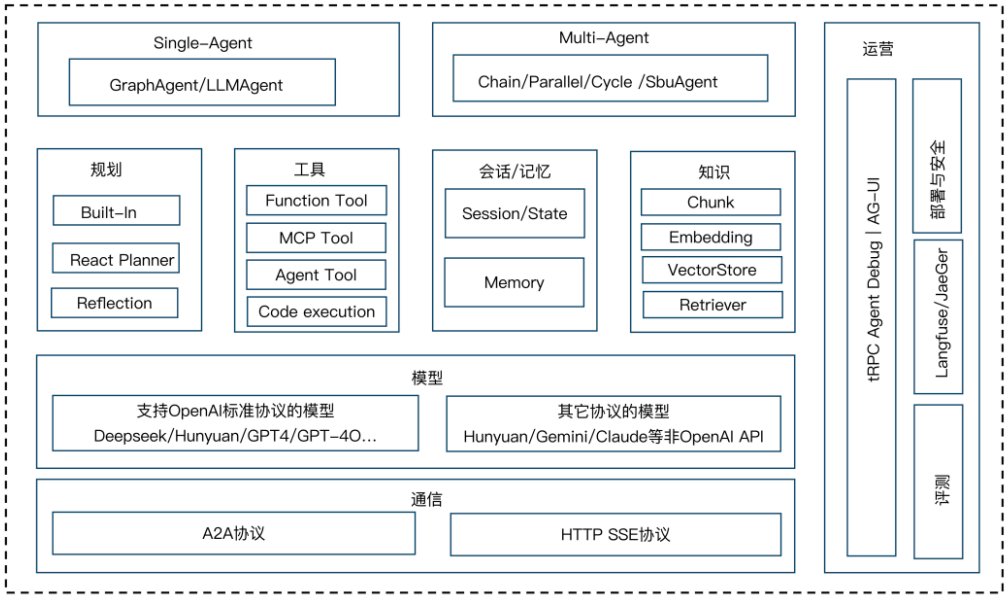
基于上面的原因，tRPC-Agent-Go的核心架构是基于自主多Agent协作的框架，同时又提供工作流编排的能力。

二、tRPC-Agent-Go框架总览

tRPC-Agent-Go（https://github.com/trpc-group/trpc-agent-go） 框架集成了LLM、智能规划器、会话管理、可观测性和丰富的工具生态系统。支持创建自主Agent和半自主Agent，具备推理能力、工具调用、子Agent协作和长期状态保持能力，为开发者提供构建智能应用的完整技术栈。

1.整体架构图

1.1 模块图



● Agent: 核心执行单元，负责处理用户输入并生成响应，提供智能推理引擎和任务编排能力，tRPC Agent支持单Agent（GraphAgent，LLMAgent）和Multi-Agent（ChainAgent，ParallelAgent，CycleAgent），并支持基于muti-agent-system进行串联。

● Runner: Agent 的执行器，负责管理Agent的生命周期管理、会话状态维护和事件流处理，串联 Session/Memory Service 等能力

● Model: Model模块提供了统一的LLM接口抽象，支持OpenAI兼容的API调用。通过标准化的接口设计，开发者可以灵活切换不同的模型提供商，实现模型的无缝集成和调用。该模块主要支持了OpenAI like接口的兼容性，已验证公司内外大多数接口。

● Tool:Tool模块提供了标准化的工具定义、注册和执行机制，使Agent能够与外部世界进行交互。支持同步调用（CallableTool）和流式调用（StreamableTool）两种模式，满足不同场景的技术需求， 提供各种工具能力FunctionTool、MCP、DuckDuckGo 等

● Session: Session模块提供了会话管理功能，用于维护Agent与用户交互过程中的对话历史和上下文信息。会话管理模块支持多种存储后端，包括内存存储和Redis存储

● Memory: 记录用户的长期记忆和个性化信息

● Knowledge: 知识管理核心组件，它实现了完整的RAG（检索增强生成）能力。该模块不仅提供了基础的知识存储和检索功能，还支持多种高级特性：

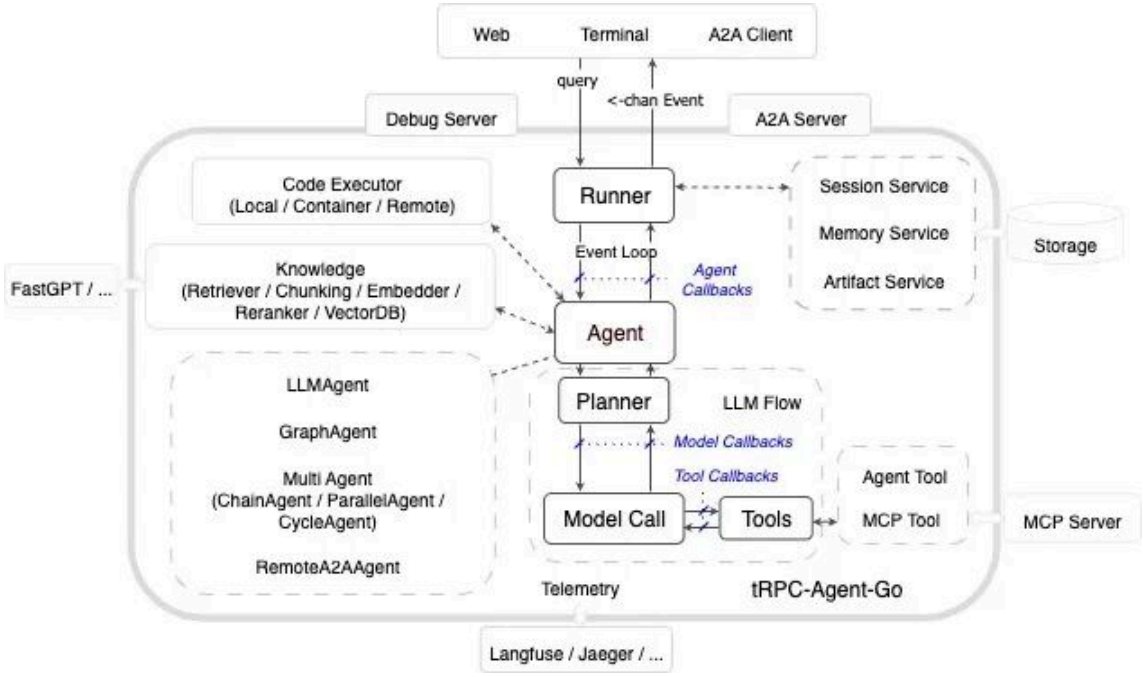
● 知识源管理：支持本地文件（Markdown、PDF、TXT等）、目录批量导入、网页抓取，智能识别输入类型。向量存储：提供内存存储（开发测试）、PostgreSQL 、pgvector、ElasticSearch。Embedding：集成OpenAI Embedding、Gemini Embedding，支持自定义模型，优化异步批处理性能。智能检索：语义相似度搜索，支持多轮对话上下文，结果重排序提升相关性。

● Planner: Planner模块为Agent提供智能规划能力，通过不同的规划策略增强Agent的推理和决策能力。支持内置思考模型、React结构化规划和自定义显式规划指导三种模式，使Agent能够更好地分解复杂任务和制定执行计划

● CodeExecutor：CodeExecutor模块为Agent提供代码执行能力，支持在本地环境或Docker容器中执行Python、Bash代码，使Agent具备数据分析、科学计算、脚本自动化等实际工作能力。

● Observability：集成OpenTelemetry标准，在Agent执行过程中自动记录详细的telemetry数据，支持全链路追踪和性能监控，内置诸多指标。

1.2 tRPC Agent整体交互流程设计



整个tRPC Agent框架采用分层设计实现：

● Runner模块是Agent的执行器和运行环境，负责Agent的生命周期管理、会话状态维护和事件流处理，对接外部web服务

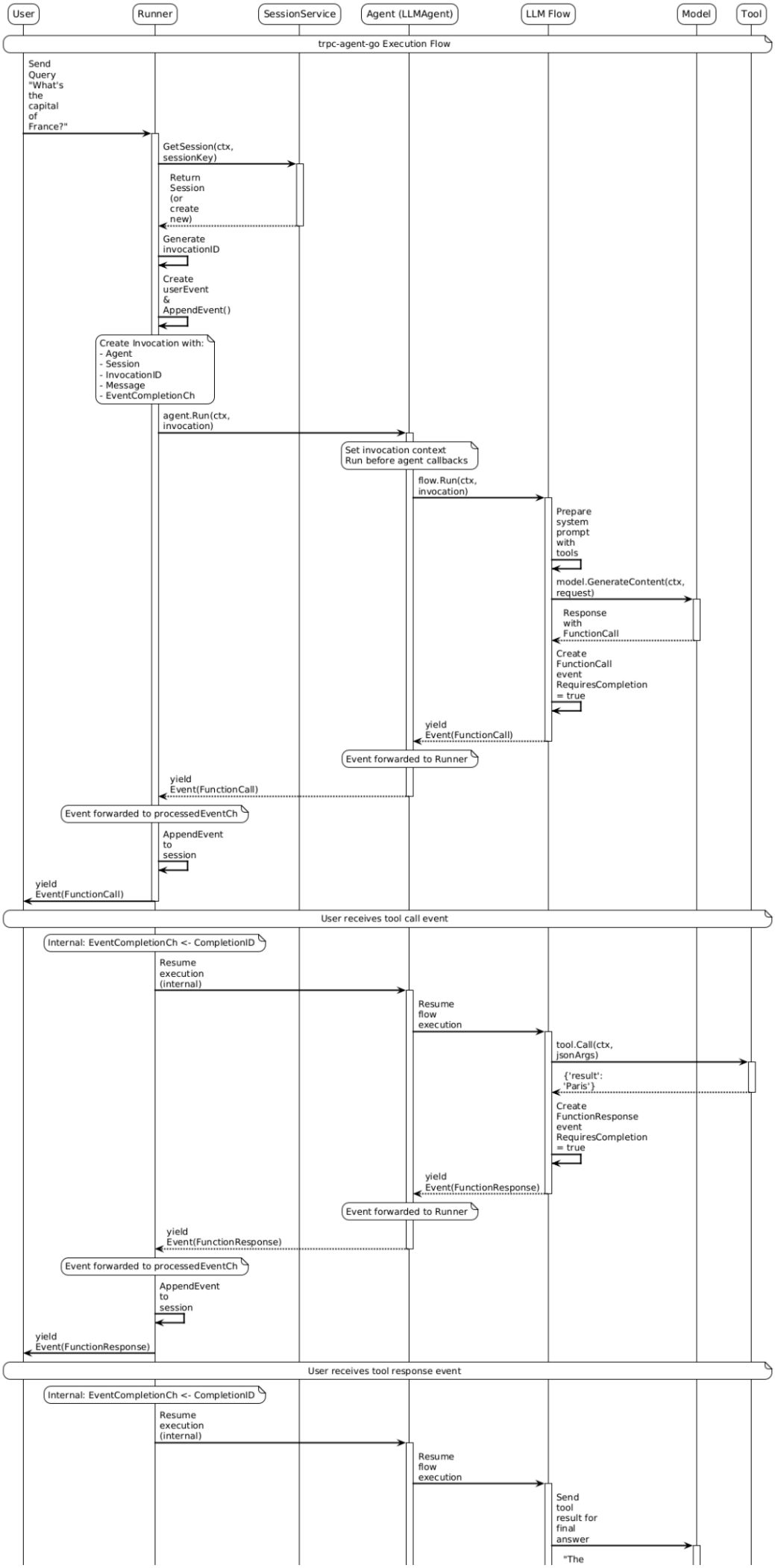
● Event模块负责Agent执行过程中的状态传递和实时通信。通过统一的事件模型，实现Agent间解耦通信和透明执行监控。

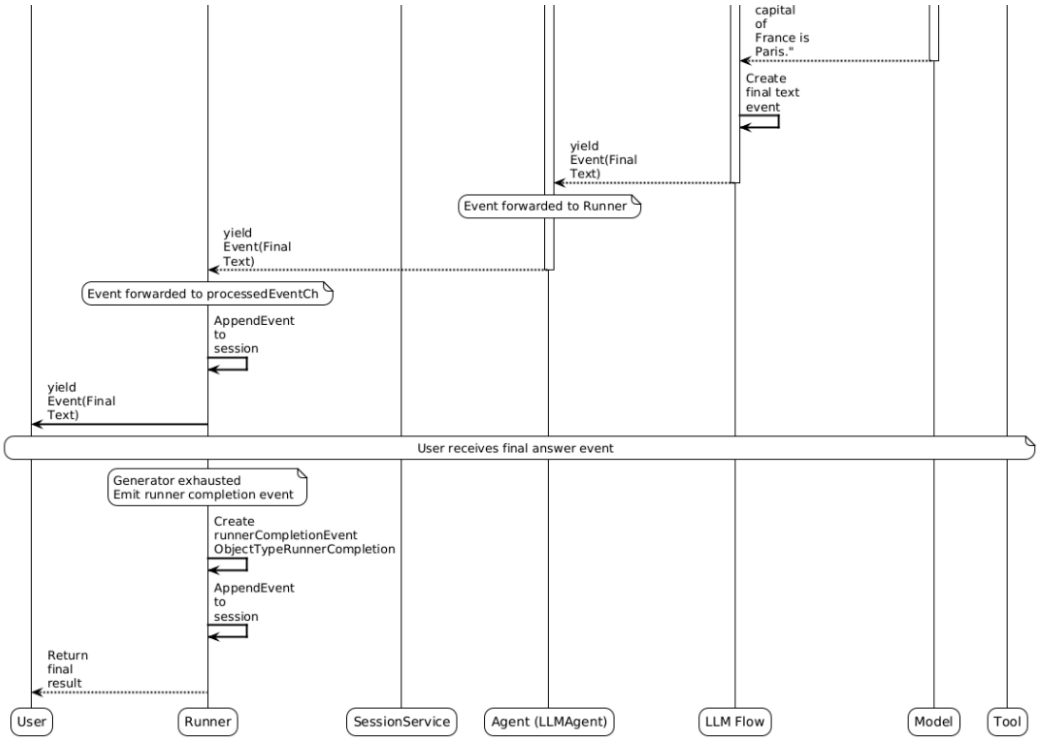
● tRPC Agent沿用tRPC的插件化设计，所有组件都能做插件化集成，tRPC Agent有内置的组件，也做了各种生态组件的集成。

● tRPC Agent的Callbacks模块还提供了一套完整的回调机制，允许在Agent执行、模型推理和工具调用的关键节点进行拦截和处理。通过回调机制，可以实现日志记录、性能监控、内容审核等功能。

1.3 时序图：

下面展示一个完整的用户和 Agent 对话的完整时序图，展示了runner、service，Agent，LLMFlow，Model，Tool之间的关系。


User
Runner
SessionService
Agent (LLMAgent)
LLM Flow
Model
Tool
trpc-agent-go Execution Flow
Send Query "What's the capital of France?"
GetSession(ctx, sessionKey)
Return Session (or create new)
Generate invocationID
Create userEvent & AppendEvent()
Create Invocation with:
- Agent
- Session
- InvocationID
- Message
- EventCompletionCh
agent.Run(ctx, invocation)
Set invocation context
Run before agent callbacks
flow.Run(ctx, invocation)
Prepare system prompt with tools
model.GenerateContent(ctx, request)
Response with FunctionCall
Create FunctionCall event RequiresCompletion = true
yield Event(FunctionCall)
Event forwarded to Runner
yield Event(FunctionCall)
Event forwarded to processedEventCh
AppendEvent to session
yield Event(FunctionCall)
User receives tool call event
Internal: EventCompletionCh <- CompletionID
Resume execution (internal)
Resume flow execution
tool.Call(ctx, jsonArgs)
{'result': 'Paris'}
Create FunctionResponse event RequiresCompletion = true
yield Event(FunctionResponse)
Event forwarded to Runner
yield Event(FunctionResponse)
Event forwarded to processedEventCh
AppendEvent to session
yield Event(FunctionResponse)
User receives tool response event
Internal: EventCompletionCh <- CompletionID
Resume execution (internal)
Resume flow execution
Send tool result for final answer
"The



三、核心模块详解

1.Model模块 – 大语言模型抽象层

Model模块提供了统一的LLM接口抽象，支持OpenAI兼容的API调用。通过标准化的接口设计，开发者可以灵活切换不同的模型提供商，实现模型的无缝集成和调用。该模块主要支持了OpenAI like接口的兼容性，已验证公司内外大多数接口。

1.1 核心接口设计

```
// Model是所有语言模型必须实现的接口
type Model interface {
    // 生成内容，支持流式响应
    GenerateContent(ctx context.Context, request *Request) (<-chan *Res

    // 返回模型基本信息
    Info() Info
}

// 模型信息结构
type Info struct {
    Name string // 模型名称
}
```

1.2 OpenAI兼容实现

框架提供了完整的OpenAI兼容实现，支持连接各种OpenAI-like接口：

```
// 创建OpenAI模型
model := openai.New("gpt-4o-mini",
    openai.WithAPIKey("your-api-key"),
    openai.WithBaseURL("https://api.openai.com/v1"), // 可自定义BaseURL
)

// 支持自定义配置
model := openai.New("custom-model",
    openai.WithAPIKey("your-api-key"),
    openai.WithBaseURL("https://your-custom-endpoint.com/v1"),
    openai.WithChannelBufferSize(512),
    openai.WithExtraFields(map[string]interface{}{
        "custom_param": "value",
    }),
)
```

1.3 支持的模型平台

当前框架支持所有提供OpenAI兼容API的模型平台，包括但不限于：

- **OpenAI** - GPT-4o、GPT-4、GPT-3.5等系列模型
- **腾讯云** - DeepSeek，Hunyuan系列
- **其他云厂商** - 提供OpenAI兼容接口的各类模型，如DeepSeek，Qwen等

Model模块的详细介绍详见：https://trpc-group.github.io/trpc-agent-go/zh/model/

2.Agent模块 - Agent执行引擎

Agent模块是tRPC-Agent-Go的核心组件，提供智能推理引擎和任务编排能力。该模块具备以下核心功能：

- **多样化Agent类型**：支持LLM、Chain、Parallel、Cycle、Graph等不同执行模式
- **工具调用与集成**：提供丰富的外部能力扩展机制

● **事件驱动架构**：实现流式处理和实时监控

● **层次化组合**：支持子Agent协作和复杂流程编排

● **状态管理**：确保长对话和会话持久化

Agent模块通过统一接口标准实现高度模块化，为开发者提供从智能对话助手到复杂任务自动化的完整技术支持。

2.1 核心接口设计

```
type Agent interface {
    // 执行Agent调用，返回事件流
    Run(ctx context.Context, invocation *Invocation) (<-chan *event.Eve

    // 返回Agent可用的工具列表
    Tools() []tool.Tool

    // 返回Agent的基本信息
    Info() Info

    // 返回子Agent列表，支持层次化组合
    SubAgents() []Agent

    // 根据名称查找子Agent
    FindSubAgent(name string) Agent
}
```

2.2 多种Agent类型

2.21 LLMAgent – 基础智能Agent

核心特点：基于LLM的智能Agent，支持工具调用、流式输出和会话管理。

● **执行方式**：直接与LLM交互，支持单轮对话和多轮会话

● **适用场景**：智能客服、内容创作、代码助手、数据分析、问答系统

● **优势**：简单直接、响应快速、配置灵活、易于扩展

```
agent := llmagent.New(
    "assistant",
```

```
    llmagent.WithModel(openai.New("gpt-4o-mini")),
    llmagent.WithInstruction("你是一个专业的AI助手"),
    llmagent.WithTools([]tool.Tool{calculatorTool, searchTool}),
)
```

2.22 ChainAgent – 链式处理Agent

核心特点: 流水线模式，多个Agent按顺序执行，前一个的输出成为后一个的输入。

● **执行方式**: Agent1 → Agent2 → Agent3 顺序执行

● **适用场景**: 文档处理流水线、数据ETL、内容审核链条

● **技术优势**: 专业分工、流程清晰、易于调试

```
chain := chainagent.New(
    "content-pipeline",
    chainagent.WithSubAgents([]agent.Agent{
        planningAgent,   // 第一步：制定计划
        researchAgent,   // 第二步：收集信息
        writingAgent,    // 第三步：创作内容
    }),
)
```

2.23 ParallelAgent – 并行处理Agent

核心特点: 并发模式，多个Agent同时执行相同任务，然后合并结果。

● **执行方式**: Agent1 + Agent2 + Agent3 同时执行

● **适用场景**: 多专家评估、多维度分析、决策支持

● **技术优势**: 并发执行、多角度分析、容错性强

```
parallel := parallelagent.New(
    "multi-expert-evaluation",
    parallelagent.WithSubAgents([]agent.Agent{
        marketAgent,      // 市场分析专家
        technicalAgent,   // 技术评估专家
        financeAgent,     // 财务分析专家
```

```
    }),
)
```

2.24 CycleAgent – 循环迭代Agent

核心特点: 迭代模式，通过多轮"执行→评估→改进"循环，不断优化结果。

● **执行方式**: 循环执行直到满足条件或达到最大轮次

● **适用场景**: 复杂问题求解、内容优化、自动调试

● **技术优势**: 自我改进、质量提升、智能停止

```
cycle := cycleagent.New(
    "problem-solver",
    cycleagent.WithSubAgents([]agent.Agent{
        generatorAgent,  // 生成解决方案
        reviewerAgent,   // 评估质量
    }),
    // 设置最大迭代次数为5，防止无限循环
    cycleagent.WithMaxIterations(5),
)
```

2.25 GraphAgent – 图工作流Agent

核心特点: 基于图的工作流模式，支持条件路由和多节点协作的复杂任务处理。

设计目的: 为了满足和兼容腾讯内部之前大多数的AI Agent应用是基于图编排框架进行开发的，方便存量用户迁移，保留已有的开发习惯。

● **执行方式**: 按图结构执行，支持LLM节点、工具节点、条件分支和状态管理

● **适用场景**: 复杂决策流程、多步骤任务协作、动态路由处理、存量图编排应用迁移

● **技术优势**: 灵活路由、状态共享、可视化流程、兼容现有开发模式

```
// 创建文档处理工作流
stateGraph := graph.NewStateGraph(graph.MessagesStateSchema())

// 创建分析工具
```

```
complexityTool := function.NewFunctionTool(
    analyzeComplexity,
    function.WithName("analyze_complexity"),
    function.WithDescription("分析文档复杂度"),
)
tools := map[string]tool.Tool{"analyze_complexity": complexityTool}

// 构建工作流图
g, err := stateGraph.
    AddNode("preprocess", preprocessDocument).          // 预处理节点
    AddLLMNode("analyze", model,
        "分析文档复杂度，使用analyze_complexity工具", tools). // LLM分析节点
    AddToolsNode("tools", tools).                       // 工具节点
    AddNode("route_complexity", routeComplexity).       // 路由决策节点
    AddLLMNode("summarize", model, "总结复杂文档", nil).  // LLM总结节点
    AddLLMNode("enhance", model, "提升简单文档质量", nil). // LLM增强节点
    AddNode("format_output", formatOutput).             // 格式化节点
    SetEntryPoint("preprocess").                        // 设置入口
    SetFinishPoint("format_output").                    // 设置出口
    AddEdge("preprocess", "analyze").                   // 连接节点
    AddToolsConditionalEdges("analyze", "tools", "route_complexity").
    AddConditionalEdges("route_complexity", complexityCondition, map[s
        "simple":  "enhance",
        "complex": "summarize",
    }).
    AddEdge("enhance", "format_output").
    AddEdge("summarize", "format_output").
    Compile()

// 创建GraphAgent并运行
graphAgent, err := graphagent.New("document-processor", g,
    graphagent.WithDescription("文档处理工作流"),
    graphagent.WithInitialState(graph.State{}),
)

runner := runner.NewRunner("doc-workflow", graphAgent)
events, _ := runner.Run(ctx, userID, sessionID,
    model.NewUserMessage("处理这个文档内容"))
```

Agent详细介绍：https://trpc-group.github.io/trpc-agent-go/zh/agent/

2.3 Multi-Agent System – 多Agent协作系统

tRPC-Agent-Go采用SubAgent机制构建多Agent系统，支持多个Agent协作处理复杂任务。

```go
// 创建专业领域Agent
marketAnalyst := llmagent.New("market-analyst",
    llmagent.WithModel(model),
    llmagent.WithInstruction("你是市场分析专家"),
    llmagent.WithTools([]tool.Tool{marketDataTool}))

techArchitect := llmagent.New("tech-architect",
    llmagent.WithModel(model),
    llmagent.WithInstruction("你是技术架构专家"),
    llmagent.WithTools([]tool.Tool{techAnalysisTool}))

// 串行协作：市场分析 → 技术评估
planningChain := chainagent.New("product-planning",
    chainagent.WithSubAgents([]agent.Agent{
        marketAnalyst, techArchitect,
    }))

// 并行协作：多专家同时评估
expertPanel := parallelagent.New("expert-panel",
    parallelagent.WithSubAgents([]agent.Agent{
        marketAnalyst, techArchitect,
    }))

// 执行多Agent协作
events, err := expertPanel.Run(ctx, &agent.Invocation{
    Message: model.NewUserMessage("分析市场，设计产品方案"),
})
```

3.Event模块 – 事件驱动系统

Event模块是tRPC-Agent-Go的事件系统核心，负责Agent执行过程中的状态传递和实时通信。通过统一的事件模型，实现Agent间解耦通信和透明执行监控。

3.1 核心特性

- **异步通信**：Agent通过事件流进行非阻塞通信，支持高并发执行
- **实时监控**：所有执行状态通过事件实时传递，支持流式处理
- **统一抽象**：不同类型Agent通过相同事件接口交互
- **多Agent协作**：支持分支事件过滤和状态追踪

3.2 核心接口

```go
// Event代表Agent执行过程中的一个事件
type Event struct {
    *model.Response      // 嵌入LLM响应的所有字段
    InvocationID string  // 本次调用的唯一标识
    Author       string  // 事件发起者（Agent名称）
    ID           string  // 事件唯一标识
    Timestamp    time.Time // 事件时间戳
    Branch       string  // 分支标识（多Agent协作）
}
```

3.3 主要事件类型

- chat.completion – LLM对话完成事件
- chat.completion.chunk – 流式对话事件
- tool.response – 工具响应事件
- agent.transfer – Agent转移事件
- error – 错误事件

3.4 Agent.Run()与事件处理

所有Agent都通过Run()方法返回事件流，实现统一的执行接口：

```go
// Agent接口定义
type Agent interface {
    Run(ctx context.Context, invocation *Invocation) (<-chan *event.Ev
}

// 创建Agent并处理事件流
agent := llmagent.New("assistant",
    llmagent.WithModel(model),
    llmagent.WithTools(tools))

```

```
events, err := agent.Run(ctx, &agent.Invocation{
    Message: model.NewUserMessage("计算 2+3 等于多少"),
})

// 实时处理事件流
for event := range events {
    switch event.Object {
    case "chat.completion.chunk":
        fmt.Print(event.Choices[0].Delta.Content)
    case "tool.response":
        fmt.Printf("\n[%s] 工具执行完成\n", event.Author)
    case "chat.completion":
        if event.Done {
            fmt.Printf("\n[%s] 最终答案: %s\n",
                event.Author, event.Choices[0].Message.Content)
        }
    case "error":
        fmt.Printf("错误: %s\n", event.Error.Message)
        return event.Error
    }
    if event.Done { break }
}
```

3.5 多Agent协作中的事件流

```
chainAgent := chainagent.New("chain",
    chainagent.WithSubAgents([]agent.Agent{
        analysisAgent, solutionAgent,
    }))

events, err := chainAgent.Run(ctx, invocation)
if err != nil {
    return err
}

for event := range events {
    switch event.Object {
    case "chat.completion.chunk":
        fmt.Print(event.Choices[0].Delta.Content)
    case "chat.completion":
        if event.Done {
            fmt.Printf("[%s] 完成: %s\n", event.Author,
                event.Choices[0].Message.Content)
```

```
        }
    case "tool.response":
        fmt.Printf("[%s] 工具执行完成\n", event.Author)
    case "error":
        fmt.Printf("[%s] 错误: %s\n", event.Author, event.Error.Message
    }
}
```

4.Invocation – Agent执行上下文

Invocation是Agent执行的核心上下文对象，封装了单次调用所需的所有信息和状态。它作为Agent.Run()方法的参数，支持事件追踪、状态管理和Agent间协作。

4.1 核心结构

```
type Invocation struct {
        Agent             Agent                    // 要调用的Agent实例
        AgentName         string                   // Agent名称
        InvocationID      string                   // 调用唯一标识
        Branch            string                   // 分支标识符（多Agent
        EndInvocation     bool                     // 是否结束调用
        Session           *session.Session         // 会话状态
        Model             model.Model              // 语言模型
        Message           model.Message            // 用户消息
        EventCompletionCh <-chan string           // 事件完成信号
        RunOptions        RunOptions               // 运行选项
        TransferInfo      *TransferInfo            // Agent转移信息
        AgentCallbacks    *Callbacks               // Agent回调
        ModelCallbacks    *model.Callbacks         // 模型回调
        ToolCallbacks     *tool.Callbacks          // 工具回调
}

type TransferInfo struct {
        TargetAgentName string // 目标Agent名称
        Message         string // 转移消息
        EndInvocation   bool   // 转移后是否结束
}
```

4.2 主要功能

- **执行上下文**：Agent标识、调用追踪、分支控制
- **状态管理**：会话历史、模型配置、消息传递
- **事件控制**：异步通信、执行选项
- **Agent协作**：控制权转移、回调机制

4.3 使用示例

```go
// 创建 Invocation 对象（高级用法）
invocation := agent.NewInvocation(
    agent.WithInvocationAgent(r.agent),
    agent.WithInvocationSession(&session.Session{ID: "session-001"}),
    agent.WithInvocationEndInvocation(false),
    agent.WithInvocationMessage(model.NewUserMessage("User input")),
    agent.WithInvocationModel(modelInstance),
)

// 直接调用 Agent（高级用法）
ctx := context.Background()
eventChan, err := llmAgent.Run(ctx, invocation)
if err != nil {
    log.Fatalf("执行 Agent 失败: %v", err)
}
```

4.4 最佳实践

- 优先使用Runner自动创建Invocation
- 框架会自动填充Model、Callbacks等字段
- 使用transfer工具实现Agent转移，避免直接设置TransferInfo

5.Planner模块 – 智能规划引擎

Planner模块为Agent提供智能规划能力，通过不同的规划策略增强Agent的推理和决策能力。支持内置思考模型、React结构化规划和自定义显式规划指导三种模式，使Agent能够更好地分解复杂任务和制定执行计划。其中React模式通过"思考–行动"循环和结构化标签，为普通模型提供显式的推理指导，确保Agent能够系统性地处理复杂任务。

5.1 核心接口设计

```go
// Planner接口定义了所有规划器必须实现的方法
type Planner interface {
    // 构建规划指令，为LLM请求添加规划相关的系统指令
    BuildPlanningInstruction(
        ctx context.Context,
        invocation *agent.Invocation,
        llmRequest *model.Request,
    ) string

    // 处理规划响应，对LLM的响应进行后处理和结构化
    ProcessPlanningResponse(
        ctx context.Context,
        invocation *agent.Invocation,
        response *model.Response,
    ) *model.Response
}
```

5.2 内置规划策略

5.21 Builtin Planner – 内置思考规划器

适用于具有原生思考能力的模型，通过配置模型参数启用内部推理机制：

```go
// 为OpenAI o系列模型配置推理强度
builtinPlanner := builtin.New(builtin.Options{
    ReasoningEffort: stringPtr("medium"), // "low", "medium", "high"
})

// 为Claude/Gemini模型启用思考模式
builtinPlanner := builtin.New(builtin.Options{
    ThinkingEnabled: boolPtr(true),
    ThinkingTokens:  intPtr(1000),
```

```
})
```

5.22 React Planner – 结构化规划器

React（Reasoning and Acting）Planner是一种AI推理模式，通过结构化标签引导模型进行"思考–行动"循环。它将复杂问题分解为四个标准化阶段：制定计划、推理分析、执行行动、提供答案。这种显式的推理过程让Agent能够系统性地处理复杂任务，同时提高决策的可解释性和错误检测能力。

5.3 集成到Agent

React Planner可以无缝集成到任何LLMAgent中，为Agent提供结构化的思考能力。集成后，Agent会自动按照React模式的四个阶段来处理用户请求，确保每个复杂任务都能得到系统性的处理。

```
// 创建带规划能力的Agent
agent := llmagent.New(
    "planning-assistant",
    llmagent.WithModel(openai.New("gpt-4o")),
    llmagent.WithPlanner(reactPlanner), // 集成规划器
    llmagent.WithInstruction("你是一个善于规划的智能助手"),
)

// Agent将自动使用规划器来：
// 1. 为复杂任务制定步骤化计划（PLANNING阶段）
// 2. 在执行过程中进行推理分析（REASONING阶段）
// 3. 调用相应工具执行具体操作（ACTION阶段）
// 4. 整合所有信息提供完整答案（FINAL_ANSWER阶段）
```

实际应用效果：使用React Planner的Agent在处理复杂查询时，会展现出明显的结构化思考特征。例如，当用户询问"帮我制定一个旅行计划"时，Agent会首先分析需求（PLANNING），然后推理最佳路线（REASONING），接着查询具体信息（ACTION），最后提供完整的旅行建议（FINAL_ANSWER）。这种方式不仅提高了回答质量，还让用户能够清楚地看到Agent的思考过程。

5.4 自定义规划器

开发者可以实现自定义规划器来满足特定需求：

```go
// 自定义Reflection规划器示例
type ReflectionPlanner struct {
    maxIterations int
}

func (p *ReflectionPlanner) BuildPlanningInstruction(
    ctx context.Context,
    invocation *agent.Invocation,
    llmRequest *model.Request,
) string {
    return `请按以下步骤进行反思式规划：
1. 分析问题并制定初始计划
2. 执行计划并收集结果
3. 反思执行过程，识别问题和改进点
4. 基于反思优化计划并重新执行
5. 重复反思-优化过程直到达到满意结果`
}

func (p *ReflectionPlanner) ProcessPlanningResponse(
    ctx context.Context,
    invocation *agent.Invocation,
    response *model.Response,
) *model.Response {
    // 处理反思内容，提取改进建议
    // 实现反思逻辑...
    return response
}

// 使用自定义规划器
reflectionPlanner := &ReflectionPlanner{maxIterations: 3}
agent := llmagent.New(
    "reflection-agent",
    llmagent.WithModel(model),
    llmagent.WithPlanner(reflectionPlanner), // 使用自定义规划器
)
```

Planner 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/planner/

6.Tool模块 – 工具调用框架

Tool模块提供了标准化的工具定义、注册和执行机制，使Agent能够与外部世界进行交互。支持同步调用（CallableTool）和流式调用（StreamableTool）两种模式，满足不同场景的技术需求。

6.1 核心接口设计

```go
// 基础工具接口
type Tool interface {
    Declaration() *Declaration  // 返回工具元数据
}

// 同步调用工具接口
type CallableTool interface {
    Call(ctx context.Context, jsonArgs []byte) (any, error)
    Tool
}

// 流式工具接口
type StreamableTool interface {
    StreamableCall(ctx context.Context, jsonArgs []byte) (*tool.StreamR
    Tool
}
```

6.2 工具创建示例

```go
// 计算器工具
calculatorTool := function.NewFunctionTool(
    func(ctx context.Context, input struct {
        Expression string `json:"expression"`
    }) (struct {
        Result float64 `json:"result"`
    }, error) {
        result, err := evaluateExpression(input.Expression)
        return struct{Result float64}{result}, err
    },
    function.WithName("calculator"),
    function.WithDescription("执行数学计算"),
)

 type weatherInput struct {
```

```
    Location string `json:"location"`
  }

 type weatherOutput struct {
    Weather string `json:"weather"`
 }

// 2. 实现流式工具函数
 func getStreamableWeather(input weatherInput) *tool.StreamReader {
     stream := tool.NewStream(10)
     go func() {
        defer stream.Writer.Close()

        // 模拟逐步返回天气数据
        result := "Sunny, 25°C in " + input.Location
        for i := 0; i < len(result); i++ {
            chunk := tool.StreamChunk{
                Content: weatherOutput{
                    Weather: result[i : i+1],
                },
                Metadata: tool.Metadata{CreatedAt: time.Now()},
            }

            if closed := stream.Writer.Send(chunk, nil); closed {
                break
            }
            time.Sleep(10 * time.Millisecond) // 模拟延迟
        }
    }()

    return stream.Reader
}

// 3. 创建流式工具
    weatherStreamTool :=           function.NewStreamableFunctionTool[
        getStreamableWeather,
        function.WithName("get_weather_stream"),
        function.WithDescription("流式获取天气信息"),
    )

// 创建多工具Agent
agent := llmagent.New(
    "multi-tool-assistant",
    llmagent.WithModel(model),
    llmagent.WithTools([]tool.Tool{
        calculatorTool,
```

```
        weatherStreamTool,
        duckduckgo.NewTool(),
    }),
)
```

6.3 MCP工具集成

框架支持各种mcp工具调用，提供多种连接方式：

```
// STDIO mcp工具
mcpToolSet := mcp.NewMCPToolSet(
    mcp.ConnectionConfig{
        Transport: "stdio",
        Command:   "python",
        Args:      []string{"-m", "my_mcp_server"},
        Timeout:   10 * time.Second,
    },
)

// SSE mcp工具
mcpToolSet := mcp.NewMCPToolSet(
    mcp.ConnectionConfig{
        Transport: "sse",
        ServerURL: "http://localhost:8080/sse",
        Timeout:   10 * time.Second,
        Headers: map[string]string{
            "Authorization": "Bearer your-token",
        },
    },
)
// streamable mcp工具
mcpToolSet := mcp.NewMCPToolSet(
    mcp.ConnectionConfig{
        Transport: "streamable_http",  // 注意：使用完整名称
        ServerURL: "http://localhost:3000/mcp",
        Timeout:   10 * time.Second,
    },
)

agent := llmagent.New("mcp-assistant",
    llmagent.WithModel(model),
```

```
33      llmagent.WithToolSets([]tool.ToolSet{mcpToolSet}))
```

Tool 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/tool/

7.CodeExecutor模块 – 代码执行引擎

CodeExecutor模块为Agent提供代码执行能力，支持在本地环境或Docker容器中执行Python、Bash代码，使Agent具备数据分析、科学计算、脚本自动化等实际工作能力。

7.1 核心接口设计

```go
// CodeExecutor是代码执行的核心接口
type CodeExecutor interface {
    ExecuteCode(context.Context, CodeExecutionInput) (CodeExecutionResu
    CodeBlockDelimiter() CodeBlockDelimiter
}

// 代码执行输入和结果
type CodeExecutionInput struct {
    CodeBlocks  []CodeBlock
    ExecutionID string
}

type CodeExecutionResult struct {
    Output      string  // 执行输出
    OutputFiles []File  // 生成的文件
}
```

7.2 两种执行器实现

7.21 LocalCodeExecutor – 本地执行器

直接在本地环境执行代码，适用于开发测试和可信环境：

```go
// 创建本地执行器
localExecutor := local.New(
    local.WithWorkDir("/tmp/code-execution"),
```

```
    local.WithTimeout(30*time.Second),
    local.WithCleanTempFiles(true),
)

// 集成到Agent
agent := llmagent.New(
    "data-analyst",
    llmagent.WithModel(model),
    llmagent.WithCodeExecutor(localExecutor), // 集成代码执行器
    llmagent.WithInstruction("你是数据分析师，可以执行Python代码"),
)
```

7.22 ContainerCodeExecutor – 容器执行器

在隔离的Docker容器中执行代码，提供更高安全性，适用于生产环境：

```
// 创建容器执行器
containerExecutor, err := container.New(
    container.WithContainerConfig(container.Config{
        Image: "python:3.11-slim",
    }),
    container.WithHostConfig(container.HostConfig{
        AutoRemove:  true,
        NetworkMode: "none",  // 网络隔离
        Resources: container.Resources{
            Memory: 128 * 1024 * 1024,  // 内存限制
        },
    }),
)

agent := llmagent.New(
    "secure-analyst",
    llmagent.WithModel(model),
    llmagent.WithCodeExecutor(containerExecutor), // 使用容器执行器
)
```

7.3 自动代码块识别

框架自动从Agent回复中提取markdown代码块并执行：

```
// Agent回复包含代码块时会自动执行:
// ```python
// import statistics
// data = [1, 2, 3, 4, 5]
// print(f"平均值: {statistics.mean(data)}")
```

支持python和bash代码

```

#### 使用示例

```go
// 数据分析Agent
dataAgent := llmagent.New(
    "data-scientist",
    llmagent.WithModel(model),
    llmagent.WithCodeExecutor(local.New()),
    llmagent.WithInstruction("你是数据科学家，使用Python标准库进行数据分析"),
)

// 用户提问，Agent自动生成并执行代码
runner := runner.NewRunner("analysis", dataAgent)
events, _ := runner.Run(ctx, userID, sessionID,
    model.NewUserMessage("分析数据: 23, 45, 12, 67, 34, 89"))

// Agent自动:
// 1. 生成Python分析代码
// 2. 执行代码获取结果
// 3. 解读分析结果
```

CodeExecutor模块使Agent从纯对话升级为具备实际计算能力的智能助手，支持数据分析、脚本自动化、科学计算等应用场景。后续支持更多的remote代码执行器

8.Runner模块 – Agent执行器

Runner模块是Agent的执行器和运行环境，负责Agent的生命周期管理、会话状态维护和事件流处理。

8.1 核心接口
```

```go
type Runner interface {
    Run(
        ctx context.Context,
        userID string,           // 用户标识
        sessionID string,        // 会话标识
        message model.Message,   // 输入消息
        runOpts ...agent.RunOption, // 运行选项
    ) (<-chan *event.Event, error)   // 返回事件流
}
```

8.2 使用示例

```go
// 步骤1: 创建Agent
agent := llmagent.New(
    "customer-service-agent",
    llmagent.WithModel(openai.New("gpt-4o-mini")),
    llmagent.WithInstruction("你是专业的客服助手"),
)

// 步骤2: 创建Runner并绑定Agent
r := runner.NewRunner(
    "customer-service-app",  // 应用名称
    agent,                   // 绑定Agent
)
ctx := context.Background()
userMessage := model.NewUserMessage("你好！")
eventChan, err := r.Run(ctx, "user1", "session1", userMessage)
if err != nil {
    panic(err)
}
for event := range eventChan {
    if event.Error != nil {
        fmt.Printf("错误: %s\n", event.Error.Message)
        continue
    }
    if len(event.Choices) >0 {
        fmt.Print(event.Choices[0].Delta.Content)
    }
}
```

Runner 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/runner/

9.Memory模块 – 智能记忆系统

Memory模块为Agent提供持久化的记忆能力，使Agent能够跨会话记住和检索用户信息，提供个性化的交互体验。

9.1 工作原理

Agent通过内置的记忆工具自动识别和存储重要信息，支持主题标签分类管理，并在需要时智能检索相关记忆。通过 AppName+UserID 实现多租户隔离，确保用户数据安全。

9.2 应用场景

适用于个人助手、客服机器人、教育辅导、项目协作等需要跨会话记忆用户信息的场景，如记住用户偏好、追踪问题解决进度、保存学习计划等。

9.3 核心接口

```
type Service interface {
    // 添加新记忆
    AddMemory(ctx context.Context, userKey UserKey, memory string, top
    // 更新现有记忆
    UpdateMemory(ctx context.Context, memoryKey Key, memory string, to
    // 删除指定记忆
    DeleteMemory(ctx context.Context, memoryKey Key) error
    // 清空用户所有记忆
    ClearMemories(ctx context.Context, userKey UserKey) error
    // 读取最近记忆
    ReadMemories(ctx context.Context, userKey UserKey, limit int) ([]*
    // 搜索记忆
    SearchMemories(ctx context.Context, userKey UserKey, query string)
    // 获取记忆工具
    Tools() []tool.Tool
}
```

```
17
18 // 数据结构
19 type Entry struct {
20     ID        string    `json:"id"`
21     AppName   string    `json:"app_name"`
22     UserID    string    `json:"user_id"`
23     Memory    *Memory   `json:"memory"`
24     CreatedAt time.Time `json:"created_at"`
25     UpdatedAt time.Time `json:"updated_at"`
26 }
27
28 type Memory struct {
29     Memory      string     `json:"memory"`
30     Topics      []string   `json:"topics,omitempty"`
31     LastUpdated *time.Time `json:"last_updated,omitempty"`
32 }
```

9.4 快速集成

```
1 llmAgent := llmagent.New(
2     "memory-assistant",
3     llmagent.WithTools(memoryService.Tools()), // ✅ 步骤1：注册工具
4 )
5
6 runner := runner.NewRunner(
7     "app",
8     llmAgent,
9     runner.WithMemoryService(memoryService), // ✅ 步骤2：设置服务
10 )
11
```

9.5 内置记忆工具

工具名称	默认状态	功能描述
memory_add	✅ 启用	添加新的记忆条目
memory_update	✅ 启用	更新现有记忆内容

memory_search	✅ 启用	根据关键词搜索记忆
memory_load	✅ 启用	加载最近的记忆记录
memory_delete	❌ 禁用	删除指定记忆条目
memory_clear	❌ 禁用	清空用户所有记忆

9.6 使用示例

```
// Agent会自动调用记忆工具:

// 记录信息: "我叫张三，住在北京"
// → memory_add("张三住在北京", ["个人信息"])

// 查询信息: "我住在哪里？"
// → memory_search("住址") → 返回相关记忆

// 更新信息: "我搬到上海了"
// → memory_update(id, "张三住在上海", ["个人信息"])
```

Memory 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/memory/

10.Session模块 – 会话管理系统

Session模块提供了会话管理功能，用于维护Agent与用户交互过程中的对话历史和上下文信息。会话管理模块支持多种存储后端，包括内存存储和Redis存储，后续会根据用户需求补齐MySql、PgSql等存储后端，为Agent应用提供了灵活的状态持久化能力。

10.1 核心特性

- **会话持久化**：保存完整的对话历史和上下文
- **多存储后端**：支持内存存储和Redis存储，内网版本可无缝接入tRPC Redis插件
- **事件追踪**：完整记录会话中的所有交互事件

10.2 会话层次结构

```
Application (应用)
├── User Sessions (用户会话)
│   ├── Session 1 (会话1)
│   │   ├── Session Data (会话数据)
│   │   └── Events (事件列表)
│   └── Session 2 (会话2)
│       ├── Session Data (会话数据)
│       └── Events (事件列表)
└── App Data (应用数据)
```

10.3 核心接口

```go
// Service定义会话服务的核心接口
type Service interface {
// CreateSession creates a new session.
        CreateSession(ctx context.Context, key Key, state StateMap, op

// GetSession gets a session.
        GetSession(ctx context.Context, key Key, options ...Option) (*

// ListSessions lists all sessions by user scope of session key.
        ListSessions(ctx context.Context, userKey UserKey, options ...

// DeleteSession deletes a session.
        DeleteSession(ctx context.Context, key Key, options ...Option)

// AppendEvent appends an event to a session.
        AppendEvent(ctx context.Context, session *Session, event *even

// Close closes the service.
        Close() error
}
```

10.4 存储后端支持

```go
// 内存存储（适用于开发测试）
sessionService := inmemory.NewSessionService()

// Redis存储（适用于生产环境）
sessionService, err := redis.NewService(
    redis.WithURL("redis://localhost:6379/0"),
)

// 内网版本使用tRPC Redis插件，同样的方式，只需额外引入内网的redis包，即可使用tRP
import _ "git.woa.com/trpc-go/trpc-agent-go/trpc/storage/redis"

sessionService, err := redis.NewService(
    redis.WithURL("redis://localhost:6379/0"),
)
```

10.5 与Runner集成

```go
// 创建Runner并配置会话服务
runner := runner.NewRunner(
    "my-agent",
    llmAgent,
    runner.WithSessionService(sessionService), // 集成会话管理
)

// 使用Runner进行多轮对话
eventChan, err := runner.Run(ctx, userID, sessionID, userMessage)
```

Session 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/session/

11.Knowledge模块 – 知识管理系统

Knowledge模块是trpc-agent-go中的知识管理核心组件，它实现了完整的RAG（检索增强生成）能力。该模块不仅提供了基础的知识存储和检索功能，还支持多种高级特性：

（1）知识源管理

a. 支持多种格式的本地文件（Markdown、PDF、TXT等）

b. 支持目录批量导入，自动处理子目录

c. 支持网页抓取，可直接从URL加载内容

d. 智能识别输入类型，自动选择合适的处理器

（2）向量存储

a. 内存存储：适用于开发和小规模测试

b. PostgreSQL + pgvector：适用于生产环境，支持持久化

c. TcVector：云原生解决方案，适合大规模部署

（3）Embedding

a. 默认集成OpenAIEmbedding模型

b. 支持自定义Embedding模型接入

c. 异步批处理优化性能

（4）智能检索

a. 基于语义的相似度搜索

b. 支持多轮对话历史上下文

c. 结果重排序提升相关性

11.1 核心接口设计

```
// Knowledge是知识管理的主要接口
type Knowledge interface {
    // Search执行语义搜索并返回相关结果
    Search(ctx context.Context, req *SearchRequest) (*SearchResult, er
}

// SearchRequest代表带上下文的搜索请求
type SearchRequest struct {
    Query     string                 // 搜索查询文本
    History   []ConversationMessage   // 对话历史用于上下文
```

```
    UserID    string                 // 用户标识
    SessionID string                 // 会话标识
}

// SearchResult代表知识搜索的结果
type SearchResult struct {
    Document *document.Document // 匹配的文档
    Score    float64            // 相关性分数
    Text     string             // 文档内容
}
```

11.2 与Agent集成

```
// 创建知识库
kb := knowledge.New(
    knowledge.WithVectorStore(inmemory.New()),
    knowledge.WithEmbedder(openai.New()),
    knowledge.WithSources([]source.Source{
        file.New([]string{"./docs/llm.md"}),
        url.New([]string{"https://wikipedia.org/wiki/LLM"}),
    }),
)

// 加载知识库
kb.Load(ctx)

// 创建带知识库的Agent
agent := llmagent.New(
    "knowledge-assistant",
    llmagent.WithModel(model),
    llmagent.WithKnowledge(kb), // 自动添加knowledge_search工具
    llmagent.WithInstruction("使用knowledge_search工具搜索相关资料来回答问题
)
```

Knowledge 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/knowledge/

12.Observability模块 – 可观测性系统

Observability模块集成OpenTelemetry标准，在Agent执行过程中自动记录详细的telemetry数据，支持全链路追踪和性能监控。框架复用OpenTelemetry标准接口，无自定义抽象层。

12.1 快速启动

```go
import (
    agentmetric "trpc.group/trpc-go/trpc-agent-go/telemetry/metric"
    agenttrace "trpc.group/trpc-go/trpc-agent-go/telemetry/trace"
)

func main() {
    ctx := context.Background()

    // 启动telemetry收集
    cleanupTrace, _ := agenttrace.Start(ctx)  // 默认localhost:4317
    cleanupMetric, _ := agentmetric.Start(ctx) // 默认localhost:4318
    defer cleanupTrace()
    defer cleanupMetric()

    // Agent执行过程将自动记录telemetry数据
    agent := llmagent.New("assistant",
        llmagent.WithModel(openai.New("gpt-4o-mini")))

    runner := runner.NewRunner("app", agent)
    events, _ := runner.Run(ctx, "user-001", "session-001",
        model.NewUserMessage("你好"))
}
```

12.2 自动记录的Trace链路

框架自动创建以下Span层次结构：

```
invocation                              # 对话顶层span
├── call_llm                           # LLM API调用
├── execute_tool calculator            # 工具调用
├── execute_tool search                # 工具调用
└── execute_tool (merged)              # 并行工具调用合并

# GraphAgent执行链路
invocation
```

```
└── execute_graph
    ├── execute_node preprocess
    ├── execute_node analyze
    │   └── run_model
    └── execute_node format
```

12.3 主要Span属性

- **通用属性**：invocation_id, session_id, event_id
- **LLM调用**：gen_ai.request.model, llm_request/response JSON
- **工具调用**：gen_ai.tool.name, tool_call_args, tool_response JSON
- **Graph节点**：node_id, node_name, node_description

12.4 配置选项

12.41 自定义端点配置

```
cleanupTrace, _ := agenttrace.Start(ctx,
    agenttrace.WithEndpoint("otel-collector:4317"))
```

12.41 自定义Metrics

```
counter, _ := metric.Meter.Int64Counter("agent.requests.total")
counter.Add(ctx, 1, metric.WithAttributes(
    attribute.String("agent.name", "assistant")))
```

Observability 模块的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/observability/

13.Debug Server – 调试服务器

Debug Server提供HTTP调试服务，兼容ADK Web UI，支持Agent执行的可视化调试和实时监控。

13.1 快速启动

```
// 步骤1: 准备Agent实例
agents := map[string]agent.Agent{
    "chat-assistant": llmagent.New(
        "chat-assistant",
        llmagent.WithModel(openai.New("gpt-4o-mini")),
        llmagent.WithInstruction("你是一个智能助手"),
    ),
}

// 步骤2: 创建Debug Server
debugServer := debug.New(agents)

// 步骤3: 启动HTTP服务器
http.Handle("/", debugServer.Handler())
log.Fatal(http.ListenAndServe(":8080", nil))
```

Debug Server 的详细介绍请参阅：https://trpc-group.github.io/trpc-agent-go/zh/debugserver/

14.Callbacks模块 – 回调机制

Callbacks模块提供了一套完整的回调机制，允许在Agent执行、模型推理和工具调用的关键节点进行拦截和处理。通过回调机制，可以实现日志记录、性能监控、内容审核等功能。

14.1 回调类型

14.11 ModelCallbacks（模型回调）

```
        _ = model.NewCallbacks().
```

```
                RegisterBeforeModel(func(ctx context.Context, req *mode
                        fmt.Printf("🌐 Global BeforeModel: processing %
return nil, nil
                }).
                RegisterAfterModel(func(ctx context.Context, req *mode
if modelErr != nil {
                                fmt.Printf("🌐 Global AfterModel: erro
                        } else {
                                fmt.Printf("🌐 Global AfterModel: proc
                        }
return nil, nil
                })

```

a. BeforeModel：模型推理前触发，可用于输入拦截、日志记录

b. AfterModel：每个输出块后触发，可用于内容审核、结果处理

14.12 ToolCallbacks（工具回调）

```
        _ = tool.NewCallbacks().
                RegisterBeforeTool(func(ctx context.Context, toolName 
                        fmt.Printf("🌐 Global BeforeTool: executing %s
return nil, nil
                }).
                RegisterAfterTool(func(ctx context.Context, toolName st
if runErr != nil {
                                fmt.Printf("🌐 Global AfterTool: %s fa
                        } else {
                                fmt.Printf("🌐 Global AfterTool: %s co
                        }
return nil, nil
                })

```

a. BeforeTool：工具调用前触发，可用于参数验证、结果模拟

b. AfterTool：工具调用后触发，可用于结果处理、日志记录

14.13 AgentCallbacks（Agent回调）

```
        _ = agent.NewCallbacks().
                RegisterBeforeAgent(func(ctx context.Context, invocati
                        fmt.Printf("🌐 Global BeforeAgent: starting %s
return nil, nil
                }).
                RegisterAfterAgent(func(ctx context.Context, invocatio
if runErr != nil {
                                fmt.Printf("🌐 Global AfterAgent: execu
                        } else {
                                fmt.Printf("🌐 Global AfterAgent: execu
                        }
return nil, nil
                })
)

```

a. BeforeAgent：Agent执行前触发，可用于权限检查、输入验证

b. AfterAgent：Agent执行后触发，可用于结果处理、错误处理

14.2 使用场景

（1）监控和日志：记录模型调用、工具使用和Agent执行过程

（2）性能优化：监控响应时间和资源使用情况

（3）安全和审核：过滤输入内容，审核输出内容

（4）自定义处理：格式化结果，重试错误，增强内容

14.3 集成示例

```
// 创建带回调的Agent
agent := llmagent.New(
    "callback-demo",
    llmagent.WithModel(model),
    llmagent.WithModelCallbacks(modelCallbacks),
    llmagent.WithToolCallbacks(toolCallbacks),
```

```
    llmagent.WithAgentCallbacks(agentCallbacks),
)

// 创建Runner并执行
runner := runner.NewRunner(
    "callback-app",
    agent,
    runner.WithSessionService(sessionService),
)

// 执行对话
events, err := runner.Run(ctx, userID, sessionID,
    model.NewUserMessage("Hello"))
```

Callbacks模块通过提供灵活的回调机制，使得Agent的行为更可控、更透明，同时为监控、审核、定制化等需求提供了强大的支持。

15.A2A集成 – Agent间通信

A2A（Agent-to-Agent）模块提供Agent间通信能力，支持将tRPC-Agent-Go的Agent快速集成到A2A协议中，实现多Agent协作以及对外暴露能力。

15.1 快速启动

```
// 步骤1: 创建Agent
agent := llmagent.New(
    "my-agent",
    llmagent.WithModel(openai.New("gpt-4o-mini")),
    llmagent.WithInstruction("你是一个智能助手"),
)

// 步骤2: 创建A2A服务器
a2aServer, err := a2a.New(
    a2a.WithAgent(agent,true),            // 绑定Agent
    a2a.WithHost("localhost:8080"), // 设置监听地址
)
if err != nil {
    log.Fatal(err)
}

// 步骤3: 启动服务器
ctx := context.Background()
```

```
if err := a2aServer.Start(ctx); err != nil {
    log.Fatal(err)
}

log.Println("A2A服务器已启动: localhost:8080")
```



四、业务实践

tRPC-Agent-Go在内部用诸多应用，在这里介绍下四个业务场景：

1. 元宝-深度写作

业务通过GraphAgent编排自动化写作流水线。A2AAgent先行调用搜索服务获取素材，随后LLM Node生成XML大纲并流式处理，最后循环调用Writer LLM节点按大纲逐步生成内容。

2. 腾讯体育-体育大聪明

战报生成：LLMAgent通过MCP工具获取数据，并以其“思考模式”分步生成回顾、摘要与标题。

体育搜索：GraphAgent驱动业务流程：LLM节点先改写查询，Planner节点规划 MCP工具调用获取信息，最终输出并流式解析为前端卡片。

多轮助手：GraphAgent实现路由，通过A2AAgent调度体育类请求至专属Agent，非体育类问题则路由至联网SubAgent 。

3. AMS广告营销-报表Agent

GraphAgent是核心：其LLM Node结合tRAG分析意图，流式生成“行动模板”并推送前端。随后根据模板调用MCP工具获取业务数据，最终组装成图表、表格等组件化报表。

4. 腾讯视频海外-翻译Agent

基于trpc-agent框架快速构建了四个专业LLMAgent（剧情提取、术语提取、术语翻译、字幕翻译）形成工作流。框架的封装回调机制将复杂逻辑内聚，对外提供简洁接口，高效完成从原始字幕到多语言终稿的转换。



五、写在最后

特别感谢腾讯各业务单元，包括腾讯元宝、腾讯视频、腾讯新闻、IMA 和 QQ 音乐的宝贵支持和生产环境验证推动框架发展。

感谢优秀的开源框架如 ADK、Agno、CrewAI、AutoGen 等的启发，让tRPC-Agent-Go得以站在巨人的肩膀上快速前行！🙏

tRPC-Agent-Gogithub：https://github.com/trpc-group/trpc-agent-go

欢迎star！

除此之外tRPC的GoAI生态还提供了A2A（https://github.com/trpc-group/trpc-a2a-go）和MCP框架的开发能力（https://github.com/trpc-group/trpc-mcp-go）也欢迎大家使用。

关注腾讯开源公众号
获取更多最新腾讯官方开源信息！

腾讯开源
腾讯官方开源动态、开发经验分享
91篇原创内容

公众号

加入微信群即可了解更多“腾讯开源新动态”



阅读 2648

留言

写留言

还没有留言，来写首评